

Array - 30 LeetCode Questions with Explanations

ARRAY — 30 LeetCode Questions (Easy → Hard) | Java

With Teaching Explanations for Each Algorithm

🎯 EASY (1-10)

1. Find Maximum Element in Array

Algorithm: Linear Scan **Explanation:** Walk through the array once, keeping a variable `max` that always holds the largest value seen so far. Start by assuming the first element is the `max`, then compare every later element against it and update whenever you find something bigger. This is the simplest possible “keep a running best value” pattern and shows up constantly in array problems. **Time Complexity:** $O(n)$ — visits every element once **Space Complexity:** $O(1)$ — only one extra variable

Pseudocode:

```
function findMax(arr):
    max = arr[0]
    for i from 1 to length(arr)-1:
        if arr[i] > max:
            max = arr[i]
    return max
```

Java Solution:

```
class Solution {
    public int findMax(int[] arr) {
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i];
        }
        return max;
    }
}
```

2. Find Minimum Element in Array

Algorithm: Linear Scan **Explanation:** Mirror image of finding the maximum — track a running `min` instead and flip the comparison. This teaches students that the same “single pass, running best” template can be reused just by changing the comparison operator. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function findMin(arr):
    min = arr[0]
    for i from 1 to length(arr)-1:
        if arr[i] < min:
```

```
        min = arr[i]
    return min
```

Java Solution:

```
class Solution {
    public int findMin(int[] arr) {
        int min = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) min = arr[i];
        }
        return min;
    }
}
```

3. Reverse an Array

Algorithm: Two Pointer Technique **Explanation:** Place one pointer at the start (left) and one at the end (right) of the array. Swap the elements they point to, then move left forward and right backward, meeting in the middle. This is the classic “two pointers moving toward each other” pattern used for reversing, palindrome checks, and more.
Time Complexity: $O(n)$ **Space Complexity:** $O(1)$ — reversed in place

Pseudocode:

```
function reverse(arr):
    left = 0, right = length(arr)-1
    while left < right:
        swap(arr[left], arr[right])
        left++, right--
```

Java Solution:

```
class Solution {
    public void reverse(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++; right--;
        }
    }
}
```

4. Linear Search

Algorithm: Linear (Brute Force) Search **Explanation:** Check every element one at a time until the target is found or the array runs out. It's the most basic search algorithm and the baseline every other search algorithm (binary search, hashing) tries to improve on. Works on unsorted data, unlike binary search. **Time Complexity:** $O(n)$
Space Complexity: $O(1)$

Pseudocode:

```
function search(arr, target):
    for i from 0 to length(arr)-1:
        if arr[i] == target: return i
    return -1
```

Java Solution:

```
class Solution {
    public int search(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
```

```

        if (arr[i] == target) return i;
    }
    return -1;
}

```

5. Sum of Array Elements

Algorithm: Linear Accumulation **Explanation:** Keep a running total and add each element to it as you iterate. This “accumulator” pattern is the foundation for many later problems like prefix sums, Kadane’s algorithm, and running averages. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function sum(arr):
    total = 0
    for each num in arr:
        total += num
    return total

```

Java Solution:

```

class Solution {
    public int sumArray(int[] arr) {
        int total = 0;
        for (int num : arr) total += num;
        return total;
    }
}

```

6. Check if Array is Sorted (LeetCode 1051 style)

Algorithm: Linear Scan / Adjacent Comparison **Explanation:** Compare each element to the one right after it. If you ever find a pair where the earlier element is bigger than the later one, the array isn’t sorted. This “look at neighbors” idea is the basis of many array-validation problems and of simple sorting algorithms like bubble sort. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function isSorted(arr):
    for i from 0 to length(arr)-2:
        if arr[i] > arr[i+1]: return false
    return true

```

Java Solution:

```

class Solution {
    public boolean isSorted(int[] arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] > arr[i+1]) return false;
        }
        return true;
    }
}

```

7. Remove Duplicates from Sorted Array (LeetCode 26)

Algorithm: Two Pointer (Slow-Fast) Technique **Explanation:** Because the array is sorted, duplicates are always next to each other. Use a slow pointer to mark where the next unique element should go, and a

fast pointer to scan ahead. Whenever fast finds a new value, copy it to slow's next spot and move slow forward. This in-place compaction technique avoids extra memory. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function removeDuplicates(arr):
    if arr is empty: return 0
    slow = 0
    for fast from 1 to length(arr)-1:
        if arr[fast] != arr[slow]:
            slow++
            arr[slow] = arr[fast]
    return slow + 1
```

Java Solution:

```
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;
        int slow = 0;
        for (int fast = 1; fast < nums.length; fast++) {
            if (nums[fast] != nums[slow]) {
                slow++;
                nums[slow] = nums[fast];
            }
        }
        return slow + 1;
    }
}
```

8. Move Zeroes (LeetCode 283)

Algorithm: Two Pointer (In-Place Partitioning) **Explanation:** Use insertPos to track where the next non-zero element belongs. Scan the array once, and every time you see a non-zero value, place it at insertPos and advance. After the pass, everything from insertPos to the end is filled with zeroes. This is a classic “stable partition” trick done without extra space. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function moveZeroes(arr):
    insertPos = 0
    for each num in arr:
        if num != 0:
            arr[insertPos++] = num
    while insertPos < length(arr):
        arr[insertPos++] = 0
```

Java Solution:

```
class Solution {
    public void moveZeroes(int[] nums) {
        int insertPos = 0;
        for (int num : nums) {
            if (num != 0) nums[insertPos++] = num;
        }
        while (insertPos < nums.length) nums[insertPos++] = 0;
    }
}
```

9. Two Sum (LeetCode 1)

Algorithm: Hashing (HashMap Lookup) **Explanation:** Instead of checking every pair (which would be $O(n^2)$), store each number's index in a HashMap as you scan. Before inserting a number, check whether its "complement" (target minus the number) is already in the map — if so, you've found the pair instantly. Trading space for speed with a hash map is one of the most important ideas in interview-style problems. **Time Complexity:** $O(n)$ **Space Complexity:** $O(n)$

Pseudocode:

```
function twoSum(arr, target):
    map = empty hashmap
    for i from 0 to length(arr)-1:
        complement = target - arr[i]
        if complement in map:
            return [map[complement], i]
        map[arr[i]] = i
```

Java Solution:

```
import java.util.HashMap;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }
            map.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }
}
```

10. Best Time to Buy and Sell Stock (LeetCode 121)

Algorithm: Greedy / Single Pass **Explanation:** Track the lowest price seen so far while scanning left to right. At each day, imagine selling at today's price — the potential profit is today's price minus the lowest price seen before today. Keep the best profit found. This greedy "remember the best opportunity so far" pattern avoids checking every buy/sell pair. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function maxProfit(prices):
    minPrice = infinity
    maxProfit = 0
    for each price in prices:
        if price < minPrice: minPrice = price
        else if price - minPrice > maxProfit:
            maxProfit = price - minPrice
    return maxProfit
```

Java Solution:

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
        for (int price : prices) {
            if (price < minPrice) minPrice = price;
            else if (price - minPrice > maxProfit) maxProfit = price
- minPrice;
        }
        return maxProfit;
    }
}
```

```
    }  
}
```

🔴 MEDIUM (11-20)

11. Second Largest Element in Array

Algorithm: Linear Scan with Two Trackers **Explanation:** Keep two running values, max and second. Whenever a new number beats max, the old max becomes the new second before max is updated. Whenever a number beats second but not max, only second updates. This extends the “running best” pattern to track the top two values in a single pass.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function secondLargest(arr):  
    max = second = -infinity  
    for each num in arr:  
        if num > max:  
            second = max  
            max = num  
        else if num > second and num != max:  
            second = num  
    return second
```

Java Solution:

```
class Solution {  
    public int secondLargest(int[] arr) {  
        int max = Integer.MIN_VALUE, second = Integer.MIN_VALUE;  
        for (int num : arr) {  
            if (num > max) {  
                second = max;  
                max = num;  
            } else if (num > second && num != max) {  
                second = num;  
            }  
        }  
        return second;  
    }  
}
```

12. Rotate Array by K Positions (LeetCode 189)

Algorithm: Reversal Algorithm **Explanation:** A clever trick: reversing the *whole* array, then reversing the first k elements, then reversing the remaining $n-k$ elements produces the same effect as rotating the array by k positions — all done in place without extra arrays. It’s a great example of how combining simple operations (reverse) can solve a harder problem (rotate) efficiently. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function rotate(arr, k):  
    k = k % length(arr)  
    reverse(arr, 0, length(arr)-1)  
    reverse(arr, 0, k-1)  
    reverse(arr, k, length(arr)-1)
```

Java Solution:

```
class Solution {  
    public void rotate(int[] nums, int k) {
```

```

        k %= nums.length;
        reverse(nums, 0, nums.length - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, nums.length - 1);
    }

    private void reverse(int[] arr, int left, int right) {
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++; right--;
        }
    }
}

```

13. Majority Element (LeetCode 169)

Algorithm: Boyer-Moore Voting Algorithm **Explanation:** The majority element appears more than $n/2$ times, so it can “outvote” everything else. Keep a candidate and a count. If count hits zero, pick the current number as the new candidate. Increase the count when you see the candidate again, decrease it otherwise. Because the majority element wins more votes than it loses, it always survives as the final candidate. This is a beautiful example of trading a hashmap-based counting solution for a clever $O(1)$ -space trick. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function majorityElement(arr):
    count = 0, candidate = null
    for each num in arr:
        if count == 0: candidate = num
        count += (num == candidate) ? 1 : -1
    return candidate

```

Java Solution:

```

class Solution {
    public int majorityElement(int[] nums) {
        int count = 0, candidate = 0;
        for (int num : nums) {
            if (count == 0) candidate = num;
            count += (num == candidate) ? 1 : -1;
        }
        return candidate;
    }
}

```

14. Maximum Subarray — Kadane’s Algorithm (LeetCode 53)

Algorithm: Kadane’s Algorithm (Dynamic Programming)

Explanation: At every position, decide: should I extend the previous subarray, or start a fresh subarray from here? Keep currSum as the best sum ending exactly at the current index — it’s either $arr[i]$ alone or $currSum + arr[i]$, whichever is larger. Track the overall best in maxSum. This is one of the most famous DP patterns and the gateway to understanding “running/local optimum vs global optimum” thinking.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function maxSubArray(arr):
    maxSum = arr[0]

```

```

currSum = arr[0]
for i from 1 to length(arr)-1:
    currSum = max(arr[i], currSum + arr[i])
    maxSum = max(maxSum, currSum)
return maxSum

```

Java Solution:

```

class Solution {
    public int maxSubArray(int[] nums) {
        int maxSum = nums[0], currSum = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currSum = Math.max(nums[i], currSum + nums[i]);
            maxSum = Math.max(maxSum, currSum);
        }
        return maxSum;
    }
}

```

15. Product of Array Except Self (LeetCode 238)

Algorithm: Prefix-Suffix Product Technique **Explanation:** For each index, the answer is (product of everything to its left) $\times$ (product of everything to its right). Compute left-side running products in a first pass, storing them in the result array. Then sweep from the right, multiplying in a running right-side product on the fly. This avoids division entirely and keeps everything $O(n)$ with $O(1)$ extra space (excluding the output array). **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$ extra (excluding output array)

Pseudocode:

```

function productExceptSelf(arr):
    n = length(arr)
    result = array of size n
    result[0] = 1
    for i from 1 to n-1:
        result[i] = result[i-1] * arr[i-1]
    rightProduct = 1
    for i from n-1 down to 0:
        result[i] *= rightProduct
        rightProduct *= arr[i]
    return result

```

Java Solution:

```

class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        result[0] = 1;
        for (int i = 1; i < n; i++) {
            result[i] = result[i-1] * nums[i-1];
        }
        int rightProduct = 1;
        for (int i = n - 1; i >= 0; i--) {
            result[i] *= rightProduct;
            rightProduct *= nums[i];
        }
        return result;
    }
}

```

16. Sort Colors — Dutch National Flag (LeetCode 75)

Algorithm: Dutch National Flag Algorithm (3-Way Partitioning)

Explanation: Named after the three stripes of the Dutch flag, this algorithm sorts an array of 0s, 1s, and 2s in a single pass using three pointers: low (boundary for 0s), mid (current element), and high (boundary for 2s). Swap 0s to the front, 2s to the back, and leave 1s in the middle — all without a general-purpose sort. It generalizes to any 3-value partitioning problem. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function sortColors(arr):
    low = 0, mid = 0, high = length(arr)-1
    while mid <= high:
        if arr[mid] == 0: swap(low, mid); low++; mid++
        else if arr[mid] == 1: mid++
        else: swap(mid, high); high--
```

Java Solution:

```
class Solution {
    public void sortColors(int[] nums) {
        int low = 0, mid = 0, high = nums.length - 1;
        while (mid <= high) {
            if (nums[mid] == 0) {
                int t = nums[low]; nums[low] = nums[mid]; nums[mid]
= t;
                low++; mid++;
            } else if (nums[mid] == 1) {
                mid++;
            } else {
                int t = nums[mid]; nums[mid] = nums[high];
nums[high] = t;
                high--;
            }
        }
    }
}
```

17. Container With Most Water (LeetCode 11)

Algorithm: Two Pointer (Greedy) **Explanation:** Start with the widest possible container (pointers at both ends). The area is limited by the shorter wall, so moving the taller pointer inward can only shrink or keep the width without any chance of increasing the limiting height — it's always better to move the shorter wall inward, hoping for a taller one. This greedy elimination lets you scan the whole search space in one pass instead of checking every pair. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function maxArea(height):
    left = 0, right = length(height)-1
    maxArea = 0
    while left < right:
        area = min(height[left], height[right]) * (right - left)
        maxArea = max(maxArea, area)
        if height[left] < height[right]: left++
        else: right--
    return maxArea
```

Java Solution:

```
class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1, maxArea = 0;
        while (left < right) {
```

```

        int area = Math.min(height[left], height[right]) *
(right - left);
        maxArea = Math.max(maxArea, area);
        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxArea;
}
}

```

18. 3Sum (LeetCode 15)

Algorithm: Sorting + Two Pointer **Explanation:** Sort the array first so duplicates sit next to each other and values grow left-to-right. Fix one number `arr[i]`, then use two pointers (`left` and `right`) on the remaining sorted subarray to find pairs that sum to `-arr[i]`. Skip over duplicate values to avoid duplicate triplets. This turns an $O(n^3)$ brute force into $O(n^2)$. **Time Complexity:** $O(n^2)$ **Space Complexity:** $O(1)$ extra (excluding output list)

Pseudocode:

```

function threeSum(arr):
    sort(arr)
    result = []
    for i from 0 to length(arr)-3:
        if i > 0 and arr[i] == arr[i-1]: continue
        left = i+1, right = length(arr)-1
        while left < right:
            sum = arr[i] + arr[left] + arr[right]
            if sum == 0:
                add [arr[i], arr[left], arr[right]] to result
                skip duplicates, left++, right--
            else if sum < 0: left++
            else: right--
    return result

```

Java Solution:

```

import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        for (int i = 0; i < nums.length - 2; i++) {
            if (i > 0 && nums[i] == nums[i-1]) continue;
            int left = i + 1, right = nums.length - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left],
nums[right]));
                    while (left < right && nums[left] ==
nums[left+1]) left++;
                    while (left < right && nums[right] ==
nums[right-1]) right--;
                    left++; right--;
                } else if (sum < 0) left++;
                else right--;
            }
        }
        return result;
    }
}

```

19. Find All Duplicates in Array (LeetCode 442)

Algorithm: Index Marking (Cyclic Sort Family) **Explanation:**

Because values are in the range $[1, n]$, each value can be mapped to an index ($\text{value} - 1$). Walk through the array and, for each value, flip the sign of the element at its corresponding index to mark "I've seen this value." If you land on an index that's already negative, that value is a duplicate. This trick uses the array itself as a hash set, achieving $O(1)$ extra space. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$ extra (excluding output list)

Pseudocode:

```
function findDuplicates(arr):
    result = []
    for i from 0 to length(arr)-1:
        index = abs(arr[i]) - 1
        if arr[index] < 0:
            add abs(arr[i]) to result
        else:
            arr[index] = -arr[index]
    return result
```

Java Solution:

```
import java.util.*;

class Solution {
    public List<Integer> findDuplicates(int[] nums) {
        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            int index = Math.abs(nums[i]) - 1;
            if (nums[index] < 0) result.add(Math.abs(nums[i]));
            else nums[index] = -nums[index];
        }
        return result;
    }
}
```

20. Subarray Sum Equals K (LeetCode 560)

Algorithm: Prefix Sum + HashMap **Explanation:** Track a running prefix sum as you scan. If $\text{sum} - k$ has appeared before as a prefix sum, it means the subarray between that earlier point and now adds up to exactly k . A HashMap stores how many times each prefix sum has occurred, so you can count all valid subarrays in one pass rather than checking every possible subarray directly. **Time Complexity:** $O(n)$ **Space Complexity:** $O(n)$

Pseudocode:

```
function subarraySum(arr, k):
    map = {0: 1}
    sum = 0, count = 0
    for each num in arr:
        sum += num
        if (sum - k) in map:
            count += map[sum - k]
        map[sum] = map.get(sum, 0) + 1
    return count
```

Java Solution:

```
import java.util.HashMap;

class Solution {
    public int subarraySum(int[] nums, int k) {
        HashMap<Integer, Integer> map = new HashMap<>();
```

```

        map.put(0, 1);
        int sum = 0, count = 0;
        for (int num : nums) {
            sum += num;
            if (map.containsKey(sum - k)) count += map.get(sum - k);
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
        return count;
    }
}

```

HARD (21-30)

21. Trapping Rain Water (LeetCode 42)

Algorithm: Two Pointer Technique **Explanation:** Water trapped above any bar is limited by the shorter of the tallest wall to its left and the tallest wall to its right. Instead of precomputing both arrays, walk two pointers inward from both ends while tracking leftMax and rightMax. Whichever side has the smaller max is the side that's "locked in" — you can safely compute trapped water there and advance that pointer. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function trap(height):
    left = 0, right = length(height)-1
    leftMax = 0, rightMax = 0, water = 0
    while left < right:
        if height[left] < height[right]:
            leftMax = max(leftMax, height[left])
            water += leftMax - height[left]
            left++
        else:
            rightMax = max(rightMax, height[right])
            water += rightMax - height[right]
            right--
    return water

```

Java Solution:

```

class Solution {
    public int trap(int[] height) {
        int left = 0, right = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (left < right) {
            if (height[left] < height[right]) {
                leftMax = Math.max(leftMax, height[left]);
                water += leftMax - height[left];
                left++;
            } else {
                rightMax = Math.max(rightMax, height[right]);
                water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }
}

```

22. Merge Intervals (LeetCode 56)

Algorithm: Sorting + Greedy Merge **Explanation:** Sort intervals by their start value so overlapping intervals become neighbors. Walk through the sorted list, keeping the last merged interval in a result

list. If the current interval's start is within the last merged interval's end, merge them by extending the end; otherwise, start a new interval. Sorting first turns a hard "any overlap" problem into a simple linear scan. **Time Complexity:** $O(n \log n)$ — dominated by the sort **Space Complexity:** $O(n)$ for the result

Pseudocode:

```
function merge(intervals):
    sort intervals by start
    result = [intervals[0]]
    for i from 1 to length(intervals)-1:
        last = result[last index]
        if intervals[i][0] <= last[1]:
            last[1] = max(last[1], intervals[i][1])
        else:
            add intervals[i] to result
    return result
```

Java Solution:

```
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> result = new ArrayList<>();
        result.add(intervals[0]);
        for (int i = 1; i < intervals.length; i++) {
            int[] last = result.get(result.size() - 1);
            if (intervals[i][0] <= last[1]) {
                last[1] = Math.max(last[1], intervals[i][1]);
            } else {
                result.add(intervals[i]);
            }
        }
        return result.toArray(new int[result.size()][]);
    }
}
```

23. Next Permutation (LeetCode 31)

Algorithm: In-Place Permutation Rearrangement **Explanation:** To find the next lexicographically larger arrangement: scan from the right to find the first place where the order "breaks" (an element smaller than the one right after it) — this is the pivot. Then find the smallest element to the right of the pivot that's still bigger than it, swap them, and reverse everything after the pivot to get the smallest possible suffix. This mirrors how permutations are ordered in a dictionary. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function nextPermutation(arr):
    i = length(arr) - 2
    while i >= 0 and arr[i] >= arr[i+1]: i--
    if i >= 0:
        j = length(arr) - 1
        while arr[j] <= arr[i]: j--
        swap(arr[i], arr[j])
    reverse(arr, i+1, length(arr)-1)
```

Java Solution:

```
class Solution {
    public void nextPermutation(int[] nums) {
        int i = nums.length - 2;
        while (i >= 0 && nums[i] >= nums[i+1]) i--;
```

```

        if (i >= 0) {
            int j = nums.length - 1;
            while (nums[j] <= nums[i]) j--;
            int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
        }
        reverse(nums, i + 1, nums.length - 1);
    }

    private void reverse(int[] arr, int left, int right) {
        while (left < right) {
            int t = arr[left]; arr[left] = arr[right]; arr[right] =
t;
            left++; right--;
        }
    }
}

```

24. Maximum Product Subarray (LeetCode 152)

Algorithm: Dynamic Programming (Track Max & Min) **Explanation:** Unlike sum, products can flip sign — a very negative running product can become the maximum if multiplied by another negative number. So track both a running maxProd and minProd ending at each index. When the current number is negative, swap max and min before updating (since multiplying by a negative flips which one is bigger). This dual-tracking idea generalizes Kadane's algorithm to handle negative numbers correctly. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function maxProduct(arr):
    maxProd = arr[0], minProd = arr[0], result = arr[0]
    for i from 1 to length(arr)-1:
        if arr[i] < 0: swap(maxProd, minProd)
        maxProd = max(arr[i], maxProd * arr[i])
        minProd = min(arr[i], minProd * arr[i])
        result = max(result, maxProd)
    return result

```

Java Solution:

```

class Solution {
    public int maxProduct(int[] nums) {
        int maxProd = nums[0], minProd = nums[0], result = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] < 0) {
                int temp = maxProd;
                maxProd = minProd;
                minProd = temp;
            }
            maxProd = Math.max(nums[i], maxProd * nums[i]);
            minProd = Math.min(nums[i], minProd * nums[i]);
            result = Math.max(result, maxProd);
        }
        return result;
    }
}

```

25. First Missing Positive (LeetCode 41)

Algorithm: Cyclic Sort / Index Placement **Explanation:** In an array of n elements, the first missing positive must be between 1 and $n+1$. So place each value v (where $1 \leq v \leq n$) at index $v-1$ by swapping, essentially sorting the array into its "correct" positions using a while loop at each index. After this pass, scan for the first index where

`arr[i] != i+1` — that's the missing number. This achieves $O(n)$ time using the array itself as storage, beating any sorting-based approach.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function firstMissingPositive(arr):
    n = length(arr)
    for i from 0 to n-1:
        while arr[i] > 0 and arr[i] <= n and arr[arr[i]-1] !=
arr[i]:
            swap(arr[i], arr[arr[i]-1])
    for i from 0 to n-1:
        if arr[i] != i+1: return i+1
    return n+1
```

Java Solution:

```
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            while (nums[i] > 0 && nums[i] <= n && nums[nums[i]-1] !=
nums[i]) {
                int temp = nums[nums[i]-1];
                nums[nums[i]-1] = nums[i];
                nums[i] = temp;
            }
        }
        for (int i = 0; i < n; i++) {
            if (nums[i] != i + 1) return i + 1;
        }
        return n + 1;
    }
}
```

26. Median of Two Sorted Arrays (LeetCode 4)

Algorithm: Binary Search on Partition Point **Explanation:** Instead of merging both arrays ($O(m+n)$), binary search for a “partition” point in the smaller array such that everything to the left of the combined partition is $\leq$ everything to the right. Once the correct partition is found (all left-side max values $\leq$ all right-side min values), the median can be read directly off the boundary. This is a much harder application of binary search — searching over *partition positions* rather than values. **Time Complexity:** $O(\log(\min(m, n)))$ **Space Complexity:** $O(1)$

Pseudocode:

```
function findMedianSortedArrays(A, B):
    if length(A) > length(B): swap(A, B)
    m = length(A), n = length(B)
    low = 0, high = m
    while low <= high:
        partitionA = (low + high) / 2
        partitionB = (m + n + 1) / 2 - partitionA
        maxLeftA = (partitionA == 0) ? -inf : A[partitionA-1]
        minRightA = (partitionA == m) ? inf : A[partitionA]
        maxLeftB = (partitionB == 0) ? -inf : B[partitionB-1]
        minRightB = (partitionB == n) ? inf : B[partitionB]
        if maxLeftA <= minRightB and maxLeftB <= minRightA:
            if (m+n) % 2 == 0:
                return (max(maxLeftA, maxLeftB) +
min(minRightA, minRightB)) / 2.0
            else:
                return max(maxLeftA, maxLeftB)
        else if maxLeftA > minRightB: high = partitionA - 1
```

```
else: low = partitionA + 1
```

Java Solution:

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) return
        findMedianSortedArrays(nums2, nums1);
        int m = nums1.length, n = nums2.length;
        int low = 0, high = m;
        while (low <= high) {
            int partitionA = (low + high) / 2;
            int partitionB = (m + n + 1) / 2 - partitionA;

            int maxLeftA = (partitionA == 0) ? Integer.MIN_VALUE :
            nums1[partitionA - 1];
            int minRightA = (partitionA == m) ? Integer.MAX_VALUE :
            nums1[partitionA];
            int maxLeftB = (partitionB == 0) ? Integer.MIN_VALUE :
            nums2[partitionB - 1];
            int minRightB = (partitionB == n) ? Integer.MAX_VALUE :
            nums2[partitionB];

            if (maxLeftA <= minRightB && maxLeftB <= minRightA) {
                if ((m + n) % 2 == 0) {
                    return (Math.max(maxLeftA, maxLeftB) +
                    Math.min(minRightA, minRightB)) / 2.0;
                } else {
                    return Math.max(maxLeftA, maxLeftB);
                }
            } else if (maxLeftA > minRightB) {
                high = partitionA - 1;
            } else {
                low = partitionA + 1;
            }
        }
        throw new IllegalArgumentException("Input arrays are not
        sorted");
    }
}
```

27. Sliding Window Maximum (LeetCode 239)

Algorithm: Monotonic Deque (Sliding Window) **Explanation:**

Maintain a deque (double-ended queue) of indices whose corresponding values are in strictly decreasing order. As the window slides, remove indices that fall out of the window from the front, and remove any indices from the back whose values are smaller than the current one (they can never be the max while the current, larger value is still in the window). The front of the deque is always the current window's maximum. This gives $O(n)$ total work instead of $O(n \cdot k)$ from recomputing the max for every window. **Time Complexity:** $O(n)$

Space Complexity: $O(k)$

Pseudocode:

```
function maxSlidingWindow(arr, k):
    deque = empty
    result = []
    for i from 0 to length(arr)-1:
        remove indices out of window from front of deque
        remove indices with smaller value than arr[i] from back
        add i to deque
        if i >= k-1: add arr[deque.front()] to result
    return result
```

Java Solution:

```

import java.util.*;

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        Deque<Integer> deque = new ArrayDeque<>();
        int n = nums.length;
        int[] result = new int[n - k + 1];
        for (int i = 0; i < n; i++) {
            while (!deque.isEmpty() && deque.peekFirst() < i - k +
1) {
                deque.pollFirst();
            }
            while (!deque.isEmpty() && nums[deque.peekLast()] <
nums[i]) {
                deque.pollLast();
            }
            deque.offerLast(i);
            if (i >= k - 1) {
                result[i - k + 1] = nums[deque.peekFirst()];
            }
        }
        return result;
    }
}

```

28. Search in Rotated Sorted Array (LeetCode 33)

Algorithm: Modified Binary Search **Explanation:** Even though the array was rotated, at least one half of any given subrange (left of mid or right of mid) is still perfectly sorted. Determine which half is sorted by comparing arr[low] with arr[mid], then check if the target lies within that sorted half's range. If it does, search there; otherwise, search the other half. This is standard binary search adapted with an extra "which side is sorted" decision. **Time Complexity:** $O(\log n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function search(arr, target):
    low = 0, high = length(arr)-1
    while low <= high:
        mid = (low+high)/2
        if arr[mid] == target: return mid
        if arr[low] <= arr[mid]:
            if arr[low] <= target < arr[mid]: high = mid-1
            else: low = mid+1
        else:
            if arr[mid] < target <= arr[high]: low = mid+1
            else: high = mid-1
    return -1

```

Java Solution:

```

class Solution {
    public int search(int[] nums, int target) {
        int low = 0, high = nums.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (nums[mid] == target) return mid;
            if (nums[low] <= nums[mid]) {
                if (nums[low] <= target && target < nums[mid]) high
= mid - 1;
                else low = mid + 1;
            } else {
                if (nums[mid] < target && target <= nums[high]) low
= mid + 1;
                else high = mid - 1;
            }
        }
    }
}

```

```

    }
    }
    return -1;
}

```

29. Largest Rectangle in Histogram (LeetCode 84)

Algorithm: Monotonic Stack **Explanation:** Maintain a stack of bar indices in increasing height order. When you encounter a bar shorter than the stack's top, it means the top bar can't extend any further right, so pop it and calculate the largest rectangle using that bar's height, with its width stretching from the new stack top (or the start) to the current position. This "resolve as soon as blocked" idea using a monotonic stack is a very common hard-problem pattern. **Time Complexity:** $O(n)$ — each bar is pushed and popped once **Space Complexity:** $O(n)$

Pseudocode:

```

function largestRectangleArea(heights):
    stack = empty, maxArea = 0
    for i from 0 to length(heights):
        h = (i == length(heights)) ? 0 : heights[i]
        while stack not empty and heights[stack.top()] >= h:
            height = heights[stack.pop()]
            width = stack.empty() ? i : i - stack.top() - 1
            maxArea = max(maxArea, height * width)
        stack.push(i)
    return maxArea

```

Java Solution:

```

import java.util.*;

class Solution {
    public int largestRectangleArea(int[] heights) {
        Deque<Integer> stack = new ArrayDeque<>();
        int maxArea = 0;
        int n = heights.length;
        for (int i = 0; i <= n; i++) {
            int h = (i == n) ? 0 : heights[i];
            while (!stack.isEmpty() && heights[stack.peek()] >= h) {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }
            stack.push(i);
        }
        return maxArea;
    }
}

```

30. Longest Consecutive Sequence (LeetCode 128)

Algorithm: HashSet Technique **Explanation:** Put all numbers into a HashSet for $O(1)$ lookups. Only start counting a sequence from a number that has no predecessor (num - 1 not in the set) — this guarantees each sequence is only counted once, from its true starting point. From there, keep checking num + 1, num + 2, ... to measure the run's length. This avoids sorting and achieves linear time by cleverly skipping numbers that aren't sequence starts. **Time Complexity:** $O(n)$ **Space Complexity:** $O(n)$

Pseudocode:

```
function longestConsecutive(arr):
    set = build hashset from arr
    longest = 0
    for each num in set:
        if (num - 1) not in set:
            length = 1
            while (num + length) in set: length++
            longest = max(longest, length)
    return longest
```

Java Solution:

```
import java.util.*;

class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int num : nums) set.add(num);
        int longest = 0;
        for (int num : set) {
            if (!set.contains(num - 1)) {
                int length = 1;
                while (set.contains(num + length)) length++;
                longest = Math.max(longest, length);
            }
        }
        return longest;
    }
}
```
